

Reinforcement Learning for Transfer-Function & Camera Control

Math and Implementation Reference

Kevin Kunkel

Universität Leipzig · Masterseminar: Optimierung von Transferfunktionen
im Volume Rendering mit Reinforcement Learning

September 2026

This is a short, math-focused companion to `architecture.typ`: it derives every formula the two RL sub-projects are built on, states each environment as a Markov decision process, and summarizes the learning algorithm and the hand-coded baseline both agents are compared against. Both base agents train **offline**, directly against a scalar geometric/radiometric metric computed from raw data — neither ever calls the VTK renderer. Two extensions build on the opacity agent specifically: Section 6 trains the same MDP continually instead of in one batch, and Section 7 replaces the automatic metric with a reward model learned from human judgments of rendered images — the first point in this document where rendering enters the RL loop at all.

Contents

1. Shared building block: the opacity metric	1
1.1. Transfer function as a Gaussian mixture	1
1.2. <code>opacity_mass</code> and <code>mass_fraction</code>	2
2. RL sub-project 1 — transfer-function opacity	2
2.1. Episode setup	2
2.2. State, action, reward	2
2.3. Result (200k timesteps, SAC, 20 held-out episodes)	3
3. RL sub-project 2 — camera viewpoint	4
3.1. View direction and target direction	4
3.2. State, action, reward	4
3.3. Result (200k timesteps, SAC, <code>ct_skull</code> , 20 held-out episodes)	4
4. Learning algorithm: Soft Actor-Critic	5
5. Baseline: hill climbing (<code>search.py</code>)	5
6. Extension: continual online learning	6
6.1. Mechanics	6
6.2. Result ($N = 50k$ steps, $E = 5k$, single env)	6
7. Extension: RLHF reward model	7
7.1. Preference collection and featurization	7
7.2. Reward model	7
7.3. Environment: one render per step, not two	7

1. Shared building block: the opacity metric

Both sub-projects reuse the same transfer-function representation and the same integral metric, so it is derived once here and referenced from both.

1.1. Transfer function as a Gaussian mixture

The transfer function is a flat vector $p \in [-1, 1]^{24}$: four peaks, six parameters each — center c_i , width w_i , height h_i , and an RGB triple — always stored normalized and mapped to real units by an affine rescale

$\text{to_range}(x, \text{lo}, \text{hi}) = \text{lo} + \frac{x+1}{2} \cdot (\text{hi} - \text{lo})$. At any Hounsfield-unit value x , peak i contributes a Gaussian bump

$$g_i(x) = \exp\left(-\frac{1}{2}\left(\frac{x - c_i}{w_i}\right)^2\right), \quad \text{contrib}_i(x) = h_i \cdot g_i(x),$$

and the four peaks are composited into one clipped total opacity curve

$$\alpha(x; p) = \text{clip}\left(\sum_{i=1}^4 \text{contrib}_i(x), 0, 1\right).$$

(Color is a contribution-weighted blend of the four peaks' RGB values, irrelevant to any RL signal and omitted here.)

1.2. opacity_mass and mass_fraction

`opacity_mass` is the exact trapezoidal integral of α over an HU sub-range $[\text{lo}, \text{hi}]$, sampled at $n = 256$ points:

$$M(p; \text{lo}, \text{hi}) = \int_{\text{lo}}^{\text{hi}} \alpha(x; p) \, dx \approx \text{trapz}(\alpha(x_1; p), \dots, \alpha(x_n; p)).$$

This is the ground-truth signal everything downstream is built on — no rendering needed. Five fixed HU bands (`TISSUE_BANDS`) partition the domain into tissues (air, fat, soft, spongy, bone). Because raw M scales with a band's width (bone's band alone is 1400 HU wide, vs. 520 HU for fat), the RL observation and reward instead use the width-normalized **mass fraction**

$$\varphi(p, t) = M(p; \text{lo}_t, \text{hi}_t) / (\text{hi}_t - \text{lo}_t) \in [0, 1],$$

an average-opacity-in-band figure that does not reward a policy simply for picking a wide tissue band.

2. RL sub-project 1 — transfer-function opacity

Can a learned continuous-control policy match the hand-coded hill-climber at nudging one tissue's opacity toward a goal? Formalized as an episodic MDP $(\mathcal{S}, \mathcal{A}, R)$ with deterministic transitions (`rl/env.py`):

2.1. Episode setup

On `reset()`, all four peaks' heights are perturbed by uniform noise ± 0.3 from `default_params()`, then a target tissue $t \in \{\text{air, fat, soft, spongy, bone}\}$ and a goal direction $d \in \{+1, -1\}$ (increase/decrease) are sampled uniformly. The target peak index is resolved once via the same `_find_or_create_peak` the command layer uses, so the agent moves the vector through the identical mechanism a real user command would.

2.2. State, action, reward

$$s = (p, \text{onehot}(t), d, \varphi(p, t)) \in \mathbb{R}^{24+5+1+1} = \mathbb{R}^{31}$$

$$a \in [-1, 1], \quad \Delta = a \cdot 0.2$$

The action scales a single scalar delta (`MAX_DELTA = 0.2`) that is added directly to the target peak's **unit** height (the height slot converted from $[-1, 1]$ to $[0, 1]$, clipped back to $[0, 1]$) — no direction sign is baked into the transition; the agent must learn to pick the sign of a itself from d in the observation:

$$h_{\text{new}}^{\text{unit}} = \text{clip}(h_{\text{old}}^{\text{unit}} + \Delta, 0, 1).$$

The reward is the signed per-step change in mass fraction toward the goal:

$$r = \text{sign}(d) \cdot (\varphi(p', t) - \varphi(p, t)), \quad \text{sign}(d) = \begin{cases} +1d = \text{increase} \\ -1d = \text{decrease} \end{cases}$$

Episodes truncate (not terminate) after 20 steps — there is no terminal state, only a fixed interaction budget, matching how a real editing session has no natural “done.”

2.3. Result (200k timesteps, SAC, 20 held-out episodes)

	Mean final φ	Mean steps to 90% of best
Trained SAC policy	0.344	3.3
Hill-climb baseline	0.347	2.7

Near-parity: on this low-dimensional, single-scalar-action problem, a well-tuned hand rule is a strong baseline. Not wired into the live chat/voice loop.

3. RL sub-project 2 — camera viewpoint

Orient the camera toward a target tissue’s centroid direction, against a cheap exact geometric proxy computed directly from the volume’s voxel data — no rendering, no learned reward model (`rl/camera_env.py`).

3.1. View direction and target direction

The camera is parameterized by azimuth/elevation angles; its unit view direction in the volume’s coordinate frame is

$$v(\text{az}, \text{el}) = \begin{pmatrix} \cos(\text{el}) \sin(\text{az}) \\ \sin(\text{el}) \\ \cos(\text{el}) \cos(\text{az}) \end{pmatrix}.$$

A target tissue’s direction is computed once per episode directly from the HU volume: mask the voxels in the tissue’s band, take their physical-space centroid, and normalize its offset from the volume’s geometric center:

$$\bar{c}_t = \text{voxel-mean}(\{x : \text{lo}_t \leq \text{HU}(x) < \text{hi}_t\}) \cdot \text{spacing}, \quad \hat{d}_t = \frac{\bar{c}_t - c_{\text{vol}}}{\|\bar{c}_t - c_{\text{vol}}\|}.$$

A candidate direction is rejected (falls back to the next tissue, ultimately bone) if its offset is below 2% of the volume’s diagonal extent — too close to the center to have a meaningful direction, distinguishing real anatomical asymmetry from grid-discretization noise.

3.2. State, action, reward

$$s = (\sin(\text{az}), \cos(\text{az}), \text{el}/85, \text{onehot}(t), A) \in \mathbb{R}^9, \quad a \in [-1, 1]^2$$

$$\text{az}' = (\text{az} + a_1 \cdot 30^\circ) \bmod 360^\circ, \quad \text{el}' = \text{clip}(\text{el} + a_2 \cdot 30^\circ, -85^\circ, 85^\circ)$$

Alignment is the cosine similarity between the current view direction and the fixed target direction (both unit vectors, so a plain dot product):

$$A(\text{az}, \text{el}) = v(\text{az}, \text{el}) \cdot \hat{d}_t \in [-1, 1], \quad r = A' - A.$$

Azimuth **wraps** (no natural bound); elevation **clamps** (avoids flipping through the poles) — the same asymmetry the manual camera commands use. 20-step truncated episodes, as in Section 2.

A real bug caught during this sub-project: The hill-climb baseline seeds its step at 30° , but the shared `resize_step` growth cap defaulted to 1.0 — correct for opacity values in $[0, 1]$, wrong by $30\times$ here. The very first accepted move collapsed the step from 30° to 1° with no way to grow back: mean final alignment **0.658**. Adding an explicit `max_step` parameter (default 1.0, so the three opacity-scale callers are unaffected) and passing `max_step=30` at this call site fixed it: **0.99998**.

3.3. Result (200k timesteps, SAC, `ct_skull`, 20 held-out episodes)

	Mean final alignment	Mean steps to 90% of best
Trained SAC policy	0.997	3.05
Hill-climb baseline	1.000	4.05

Both reach near-perfect alignment — the alignment landscape is a smooth, unimodal dot product against a fixed direction, easy for either method. The trained policy’s edge is convergence speed (fewer renders needed in an interactive tool), not final quality. The reward has **no occlusion model** — it is a directional proxy for “facing the tissue,” not “can see it unobstructed.”

4. Learning algorithm: Soft Actor-Critic

Both agents are trained with `stable-baselines3`'s SAC (MlpPolicy, default two-hidden-layer-of-256 network, 4 parallel envs via `make_vec_env`, `gamma = 0.99` default). SAC is an off-policy, maximum-entropy actor-critic method: alongside expected return, the policy π is trained to maximize its own entropy $\mathcal{H}$, trading a bit of reward for exploration robustness —

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_t \gamma^t (r_t + \alpha_{\text{ent}} \mathcal{H}(\pi(\cdot | s_t))) \right],$$

with twin clipped Q-networks (reducing value overestimation) and an automatically-tuned entropy coefficient α_{ent} (both SB3 defaults, not overridden here). Both environments are pure Python/NumPy with no rendering in the loop, so training throughput is bounded by CPU, not GPU or I/O.

5. Baseline: hill climbing (`search.py`)

The comparison point for both sub-projects, and the system's actual production optimizer:

$$\text{propose_step}(p, i, s, \delta) : h_i^{\text{unit}} \leftarrow \text{clip}(h_i^{\text{unit}} + s \cdot \delta, 0, 1)$$

$$\text{resize_step}(\delta, \text{accepted}, \delta_{\text{max}}) = \begin{cases} \min(1.2\delta, \delta_{\text{max}}) & \text{accepted} \\ 0.5\delta & \text{rejected} \end{cases}$$

— grow the step $1.2 \times$ on acceptance (capped at a domain-appropriate δ_{max}), halve it on rejection. Acceptance for a relative increase/decrease command is decided by `evaluate.objective`: let $\Delta_t = \pm(M(p', \text{target}) - M(p, \text{target}))$ (sign matching the requested direction). The step is accepted iff it actually moved the target band the right way **and** no other tissue's mass moved by more — a collateral-damage check with no RL analogue:

$$\text{accept} \Leftrightarrow \Delta_t > \varepsilon \wedge \max_{t' \neq t} |M(p', t') - M(p, t')| \leq \Delta_t, \quad \varepsilon = 10^{-4}.$$

Both `rl/eval.py` and `rl/camera_eval.py` measure **steps to 90% of best** generically: given the trajectory's start value f_0 and the best value either method reached across the pair, the step index at which f first crosses $f_0 + 0.9(f_{\text{best}} - f_0)$ (or the symmetric case for a decreasing goal).

Scope: Both base agents are trained and evaluated entirely offline against the metrics above. The camera policy is not reachable from the live chat/voice loop; the opacity policy now is, as frozen inference (`server.py`'s policy search toggle, via `rl/serve.py`) — see Section 6 and Section 7 below for the two ways training itself is being extended. See `architecture.typ` for the full system this feeds into.

6. Extension: continual online learning

`rl/train.py` trains once, offline, in a single 200k-step batch across 4 parallel environments, evaluated once at the end. `rl/online_train.py` trains the **identical** MDP from Section 2 — same state, action, reward — but as one continuous stream on a single environment, checkpointing held-out performance periodically instead of only at the end, so the metric’s improvement over training is directly observable.

6.1. Mechanics

Training proceeds in chunks of `eval_interval` steps. Each chunk is a plain SB3 call — no custom replay-buffer or gradient-step code — so the online variant’s **learning** mechanics are identical to the offline agent’s; only the scheduling around it differs:

```
for  $k = 1, \dots, N/E$ : model.learn(total_timesteps :  $E$ , reset_num_timesteps : False)
```

where N is the total step budget and E the eval interval. After each chunk, the current policy is replayed (deterministically) against the same 20 held-out episodes Section 2’s offline evaluation uses, and the mean final φ and mean steps-to-90% are logged — the exact same two statistics the offline result table reports, making the two directly comparable.

6.2. Result ($N = 50\text{k steps}$, $E = 5\text{k}$, single env)

Steps	Mean final φ	Mean steps to 90%
5,000	0.199	17.35
10,000	0.281	16.00
15,000	0.347	9.35
20,000	0.347	5.15
25,000	0.346	3.80
30,000	0.346	3.30
35,000	0.344	3.30
40,000	0.345	3.25
45,000	0.346	3.20
50,000	0.345	3.20

Note: 50k single-env steps are 25% of the offline agent’s 200k-step budget (SB3’s `total_timesteps` already counts across all parallel envs, so this is a direct comparison, not one that needs a $\times 4$ correction). The online run lands at 0.345 — almost exactly between the offline policy (0.344) and hill-climb baseline (0.347) — while steps-to-90%-of-best falls steadily from 17.4 to 3.2: a clean, converging curve on a quarter of the budget, not a coincidental match.

7. Extension: RLHF reward model

φ (Section 2) measures only whether the TF curve moved the way a command literally asked — never whether the resulting **image** looks good. This extension trains a small reward model from human judgments of rendered before/after pairs, then trains RL against that learned reward instead of φ ’s exact delta — the RLHF pattern, and the first point in this document where the VTK renderer enters the RL loop at all.

7.1. Preference collection and featurization

`render.features(rgb)` (already used elsewhere for logging) reduces a rendered frame to four cheap scalars:

$$f(x) = (\text{mean}(x), \text{std}(x), \text{coverage}(x), \text{entropy}(x)) \in \mathbb{R}^4.$$

`rl/collect_preferences.py` samples cases the same way Section 2’s episodes are sampled, applies one random action, renders before/after, and asks a human for a label $y \in \{0, 1\}$ (better/worse). Each labeled example is reduced to a 10-dimensional feature vector — the same information a policy would have available when it takes that step:

$$z = (f(\text{after}) - f(\text{before}), \text{onehot}(t), d) \in \mathbb{R}^{4+5+1} = \mathbb{R}^{10}.$$

7.2. Reward model

A small MLP $R_\theta : \mathbb{R}^{10} \rightarrow \mathbb{R}$ ($10 \rightarrow 16 \rightarrow 1$, ReLU) is trained on labeled pairs (z_i, y_i) via binary cross-entropy on the sigmoid of its output:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(R_\theta(z_i)) + (1 - y_i) \log(1 - \sigma(R_\theta(z_i)))].$$

At inference, the reward handed to RL is the sigmoid output remapped to a signed range matching φ ’s reward shape:

$$\tilde{r}(z) = 2\sigma(R_\theta(z)) - 1 \in [-1, 1].$$

7.3. Environment: one render per step, not two

`RewardModelTFEnv` subclasses `TFEnv` from Section 2 unchanged — same sampling, same action, same **observation** (which still includes the free φ value) — and overrides only the reward: `step()` renders the new state, computes $\tilde{r}$ from Δf against the **previous** step’s rendered features, then caches this step’s features as next step’s “before” — so a rollout of k steps costs $k + 1$ renders, not $2k$. Measured directly on the synthetic phantom (96³, GPU raycast): **109 ms per render**, which is why this extension’s step budget (2,000) is two orders of magnitude smaller than the automatic-metric agents’ — every step now does real work a φ -based step never had to.

Scope: Deliberately framed as a single-rater pilot ($n \approx 50$ labels), not a generalizable result — reward-model train/val accuracy is reported honestly regardless of value, per the design’s explicit methodological stance. All four modules (`rl/collect_preferences.py`, `rl/reward_model.py`, `rl/reward_model_env.py`, `rl/reward_model_train.py`) are implemented and tested; the pipeline has not yet been run against real collected labels, since that step needs a live human and cannot run unattended. Camera-viewpoint RLHF and live chat-UI wiring for either extension in this section are both out of scope for now.